

ASSIGNMENT- DELTA X

OOP Case 1 - Feet on Dashboard Detector

**A Hybrid Pose-Based Human Activity Recognition System using
YOLOv8-Pose and Bidirectional LSTM**

Submitted by (30/05/26)

Hemnath N

hemnath0805@gmail.com

ABSTRACT

The thesis presents a hybrid out-of-position (OOP) detection system targeting Euro NCAP Case 1, in which a front passenger rests one or both feet on the vehicle dashboard. The system combines YOLOv8-Pose for real-time human body keypoint extraction from infrared in-cabin video with a Bidirectional Long Short-Term Memory (BiLSTM) network for sequence-based activity classification.

A body-relative normalisation scheme ensures that the feature representation is invariant to camera distance and passenger body size. A rule-based heuristic provides a robust fallback when trained model weights are unavailable. A hysteresis state machine suppresses temporal flickering and produces stable per-frame annotations. The complete pipeline is validated on infrared video footage captured inside a Hyundai Santa Fe using an Intensirics fisheye IR camera.

Keywords: Out-of-Position Detection, YOLOv8-Pose, Bidirectional LSTM, Human Activity Recognition, Euro NCAP, In-Cabin Safety Monitoring, Infrared Video, Pose Estimation

TABLE OF CONTENTS

No.	Section Title
	Cover Page
	Abstract
	Table of Contents
1	INTRODUCTION
1.1	What is Euro NCAP?
1.2	What is Out-of-Position (OOP)?
1.3	Why Does This Matter?
1.4	Scope of The Thesis
2	PROBLEM STATEMENT

2.1	Input Data
2.2	Required Output
2.3	Key Challenges
3	SYSTEM OVERVIEW
3.1	Approach Summary
3.2	The Four Main Stages
3.3	Two-Layer Detection Design
3.4	Full Detection Pipeline
4	METHODOLOGY
4.1	Camera and Input Processing
4.2	Pose Estimation - YOLOv8-Pose
4.3	Passenger Identification
4.4	Feature Engineering - Body-Relative Normalisation
4.5	Human Activity Recognition - Bidirectional LSTM

4.6 Rule-Based Heuristic Fallback

4.7 Temporal Smoothing - Hysteresis State Machine

5 PROJECT STRUCTURE

5.1 Folder Structure

5.2 File Descriptions

6 HOW TO RUN - REPRODUCTION GUIDE

6.1 Requirements

6.2 Quick Start - One Click (Recommended)

6.3 Manual Step-by-Step (Alternative)

6.4 Custom Paths (Optional)

6.5 Using a Larger Model (Optional)

6.6 Expected Output

7 RESULTS AND KEY FINDINGS

7.1 Output Video Summary

7.2	Detection Behaviour Observed
7.3	Key Findings
7.4	Where the System Works Well
7.5	Where the System Struggles
8	ASSUMPTIONS MADE
9	LIMITATIONS
10	FUTURE IMPROVEMENTS
10.1	Extend to OOP Case 2 - Leaning Forward
10.2	Larger YOLOv8 Model for Better Keypoint Accuracy
10.3	Real-Time Inference Support
10.4	Larger and More Diverse Training Dataset
10.5	Confidence Score Output
10.6	Right-Hand Drive Vehicle Support
10.7	Automatic Threshold Calibration

10.8 Multi-Passenger Support

REMARKS

REFERENCES

1. INTRODUCTION

1.1 What is Euro NCAP?

Euro NCAP (European New Car Assessment Programme) is an independent vehicle safety rating organisation. It tests and rates new cars through a set of standardised crash tests and safety assessments. Car manufacturers use these ratings to improve the safety of their vehicles before they reach the public.

1.2 What is Out-of-Position (OOP)?

Out-of-Position (OOP) refers to a situation where a vehicle occupant is not sitting in their normal, safe seating posture. This is dangerous because modern vehicles are designed with airbags and seatbelts that work correctly only when the occupant is in the expected position. An OOP occupant can be seriously injured by the very safety systems designed to protect them.

Euro NCAP defines two OOP cases:

- Case 1 - The front passenger places one or both feet on the dashboard.
- Case 2 - The driver or front passenger leans forward and is in close proximity to the dashboard.

The thesis focuses exclusively on OOP Case 1 - Feet on Dashboard.

1.3 Why Does This Matter?

When a passenger rests their feet on the dashboard, their legs are positioned directly in front of the airbag deployment zone. In a collision, the airbag deploys at extremely high speed and force. This can cause severe injuries to the legs, hips, and spine injuries that would not have occurred if the passenger was seated normally.

Detecting the posture automatically and in real time is therefore a critical capability for next-generation in-cabin safety monitoring systems.

1.4 Scope of The Thesis

The report describes the complete solution developed to detect OOP Case 1 from IR in-cabin video footage. It covers:

- The methodology and tools used.
- The full detection pipeline.
- How to reproduce the results.
- Key findings, assumptions, and limitations.
- Suggestions for future improvements

2. PROBLEM STATEMENT

2.1 Input Data

The system operates on infrared (IR) in-cabin videos recorded inside a Hyundai Santa Fe. The videos are organised into three profiles, each of which contains the same front passenger across all its recordings. Every profile contains positive videos (files prefixed positive_*.mp4) and negative videos (files prefixed negative_*.mp4). All recordings were captured using the same fisheye IR camera mounted at the rear-view mirror position, and the camera's intrinsic parameters focal length, principal point, and distortion coefficients are provided.

The expected input folder structure is as follows:

```
input/  
  profile1/  
    positive_1.mp4  
    positive_2.mp4  
    negative_1.mp4  
    negative_2.mp4  
  profile2/  
    positive_1.mp4  
    positive_2.mp4  
    negative_1.mp4  
    negative_2.mp4  
  profile3/  
    positive_1.mp4  
    negative_1.mp4
```

2.2 Required Output

For every frame of every input video, the system must produce a binary per-frame classification:

- POSITIVE - the front passenger has one or both feet on the dashboard.
- NEGATIVE - the front passenger is in normal seating posture.

The output is an annotated copy of each video with a clearly visible label on every frame, written to a mirrored output folder structure.

2.3 Key Challenges

The following challenges shape the design of the system:

- Fisheye lens distortion: the 170° ultra-wide lens produces a heavily curved image that distorts body proportions and positions.
- Infrared grayscale video: the camera outputs a single-channel IR image, not a standard RGB image, which most pose models are not natively designed for.
- Multiple people in frame: both the driver and the passenger appear in the video; the system must reliably identify and focus only on the front passenger.
- Per-frame detection required: a video-level label is insufficient; every individual frame must be classified correctly.
- Temporal flickering: a naive frame-by-frame classifier produces unstable labels; smoothing is required.
- Class imbalance: even in positive videos, only some frames show the OOP posture; the dataset has far more normal frames than OOP frames.

- Varied postures: the foot may be partially raised, fully raised, or at different angles; the system must generalise across all of these.

3. SYSTEM OVERVIEW

3.1 Approach Summary

The solution uses a hybrid approach that combines deep learning-based pose estimation, a trained neural network classifier, and a rule-based fallback all tied together with temporal smoothing to produce stable, reliable per-frame detections. The system does not analyse raw pixels directly. Instead, it first extracts the human body's key joint positions from each frame, converts them into a compact set of body relative measurements, and then classifies those measurements using a trained model. This makes the system lightweight, fast, and robust to changes in lighting, camera distance, and passenger size.

3.2 The Four Main Stages

Stage	What Happens
1. Pre-processing	Raw IR frame is converted to a format the pose model can read.
2. Pose Estimation	YOLOv8-Pose detects all people and extracts 17 body keypoints per person.
3. OOP Classification	Body measurements are fed into a Bidirectional LSTM to decide POSITIVE or NEGATIVE.
4. Post-processing	A state machine smooths the output and the result is drawn onto the frame.

3.3 Two-Layer Detection Design

The system is designed with two layers of detection so it never fails completely:

- Primary layer - Bidirectional LSTM classifier trained on body keypoint sequences. The main detector handles all complex and ambiguous postures.
- Fallback layer - A simple rule-based heuristic that activates automatically if the trained model is not available. It checks ankle position relative to the dashboard zone using camera geometry.

3.4 Full Detection Pipeline

Figure 1 below illustrates the complete step-by-step journey of a single video frame from the raw IR camera input all the way to the final annotated output frame.

The figure is displayed in the next page.

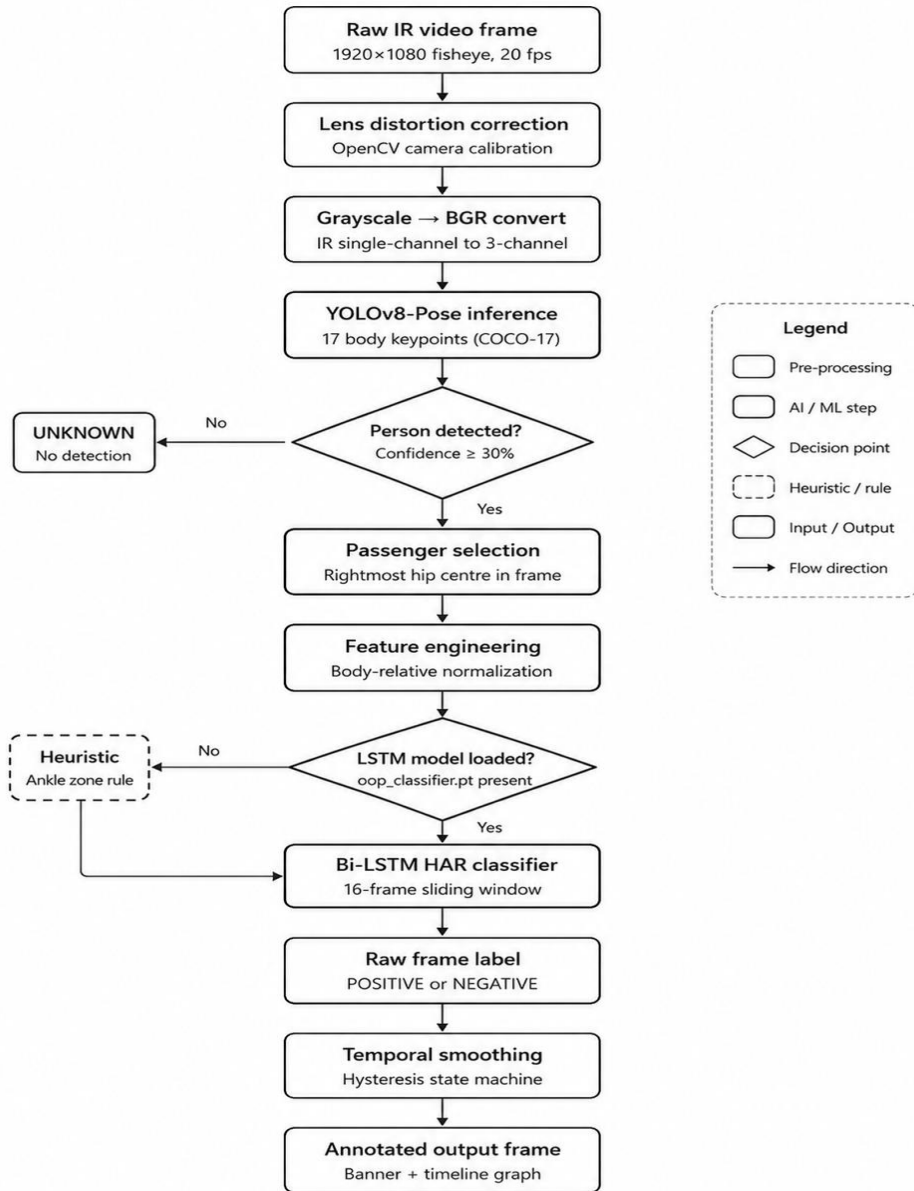


Figure 1: OOP Feet-on-Dashboard Detection Pipeline complete frame-by-frame flow

4. METHODOLOGY

4.1 Camera and Input Processing

The videos were recorded using an IntenSirics IR camera mounted at the rear-view mirror position, facing rearward into the cabin. Key camera specifications are summarised in Table 2.

Property	Value
Sensor	STMicro VG1762
Resolution	1920 × 1080
Frame Rate	20 fps (IR mode)
Lens Angle of View	170° Diagonal / 170° Horizontal / 106.3° Vertical
IR LED Wavelength	940 nm
Output Format	YUV422 8-bit

Table 2: Camera specifications for the IntenSirics IR fisheye camera.

The camera produces a single-channel infrared grayscale image. Since YOLOv8-Pose expects a 3-channel BGR image, the grayscale frame is replicated across all

three channels before inference. No colour information is lost because the original image has none as it is purely IR.

The 170° fisheye lens produces noticeable image distortion. The camera matrix and distortion coefficients are stored in config.py and can be applied via OpenCV. However, after testing, distortion correction was kept off by default because the fisheye distortion does not significantly affect body keypoint detection for the cabin geometry, and applying it introduces heavy black borders that reduce the usable frame area.

1.2 전체 사양

구분	Item	Specification
Module	Module Size	67.0mm x 32.0mm x 41.0mm(커넥터 포함)
	Resolution @ fps	1920(H) * 1080(V) @40fps(RGB20fps, IR20fps)
	Output Format	LVDS(POC)
	Interface	I2C
	Connector	FAKRA
	Operation Voltage	6V~15V
	Operation Temperature	-30°C ~ +85°C
	Storage Temperature	-40°C ~ +125°C
Image Sensor Feature	Sensor Part No.	STMicro_VG1762_1/2.5"
	Optical Size	1/2.5"
	Pixel Size	3.2um(H) * 3.2um(V)
ISP	ISP Part No.	GEO_GW5410
	output format	• YUV422@8bit with external sync or embedded sync
Serializer	ISP Part No.	Maxim_MAX9295A
	Input format	Supports RAW8/10/12/14/16/20, RGB565/666/888, YUV422 8/10-bit,
Lens Feature	Image Height(mm)	7.05(D) x 6.14(H) x 3.46(V)
	F.NO.	2.2±5%
	Angle of View	D : 170.0°±5%, H : 170.0°±5%, V : 106.3°±5%
	BFL	3.49
	TTL	18.01±0.5mm
	Thread	M12 x P0.5
IRLED	IRLE Part No.	OSRAM SFH4726AS A01
	Peak wavelength	940nm
	Half angle	75°
	IRLED POWER	12V~20V

Feet on dashboard



The pictures above are for illustration purposes only.

The system is expected to detect three foot positions on the fascia. Foot positions on the fascia are inboard, along the seat centreline and outboard will be evaluated.

Close proximity to the airbag



The pictures above are for illustration purposes only.

The system is expected to detect where the passenger is leaning forward and in close proximity to the fascia, independent of the position of the front seat.

4.2 Pose Estimation - YOLOv8-Pose

Pose estimation is the process of detecting a human body in an image and locating specific joint positions on it. The solution uses YOLOv8-Pose by Ultralytics. For every frame, YOLOv8-Pose detects all people visible in the frame and returns 17 body keypoints per person in the COCO-17 format, covering nose, eyes, ears, shoulders, elbows, wrists, hips, knees, and ankles. Each keypoint is accompanied by its X and Y pixel position and a confidence score between 0 and 1. Only keypoints with a confidence score of 0.30 or above are used.

The main reasons YOLOv8-Pose was chosen over MediaPipe were its ability to handle multiple individuals in a single frame, which is crucial in the situation because both the driver and the passenger are visible at the same time, and its dependability on IR grayscale video following a straightforward BGR conversion. MediaPipe performs poorly on infrared footage and is mostly intended for single-person RGB video.

4.3 Passenger Identification

Since both the driver and the passenger appear in the frame, the system must identify which detected person is the front passenger. The strategy used is Hip-Centre Selection for each detected person, the horizontal centre of their hips is calculated, and the person whose hip centre is furthest to the right of the frame is selected as the passenger. The works because in a left-hand drive vehicle viewed from the rear-view mirror, the front passenger always sits on the right side of the frame. If no person meets the minimum confidence threshold, the frame is labelled UNKNOWN and skipped.

4.4 Feature Engineering - Body-Relative Normalisation

Raw pixel coordinates cannot be used directly as model inputs because they change depending on how far the passenger sits from the camera or how tall they are. Instead, the keypoints are converted into body-relative normalised coordinates. The midpoint between the left and right hips becomes the origin (0, 0), and the distance between the shoulder midpoint and the hip midpoint serves as the scale factor. It means every measurement is expressed as how far a given joint is from the hip relative to torso size, making the features completely independent of camera distance and body size. The 10 keypoints used left and right shoulder, elbow, hip, knee, and ankle each contribute three values (normalised X, normalised Y, confidence), yielding a final feature vector of 30 values per frame.

4.5 Human Activity Recognition - Bidirectional LSTM

LSTM is the core intelligence of the system. The method follows the Human Activity Recognition (HAR) approach classifying a sequence of body movements over time rather than a single static frame. Looking at 16 consecutive frames (approximately 0.8 seconds at 20 fps) gives the model enough context to make a confident decision, because a single frame cannot always tell whether a foot is on the dashboard the ankle may be mid-movement, partially visible, or at an ambiguous height.

The 30-dimensional feature vectors are grouped into windows of 16 consecutive frames, each window forming one input sample for the LSTM. The model architecture is summarised in Table 3.

Component	Detail
Input	(16 frames $\times$ 30 features)
LSTM layers	2 layers, hidden size 64, bidirectional
Dropout	0.3 between LSTM layers, 0.2 before output
Output head	Linear (128 $\rightarrow$ 64) $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ Linear (64 $\rightarrow$ 1)
Output	Single logit — sigmoid > 0.5 = POSITIVE

Table 3 *Bidirectional LSTM architecture.*

The model is trained using the Adam optimiser (learning rate 0.001, weight decay $1e-4$) with Binary Cross-Entropy loss and a weighted positive class to handle class imbalance. Training runs for 80 epochs with a batch size of 64, a ReduceLROnPlateau learning rate scheduler (patience 5, factor 0.5), and gradient clipping with a maximum norm of 1.0.

4.6 Rule-Based Heuristic Fallback

If the trained LSTM model file (oop_classifier.pt) is not found, the system automatically falls back to a geometric rule based purely on the camera's physical geometry. The rule checks three conditions on the ankle's normalised vertical position, as shown in Table 4. All three conditions must be true for a frame to be

labelled POSITIVE. These thresholds are derived from the camera's physical mounting position and are not tuned against individual videos.

Check	Threshold	Meaning
Ankle is above floor level	$\text{ankle_y} / H < 0.92$	Foot is not resting on the floor.
Ankle is in dashboard zone	$\text{ankle_y} / H > 0.78$	Foot is not raised toward the ceiling.
Ankle is at or above dashboard	$\text{ankle_y} / H < 0.87$	Foot is at dashboard height.

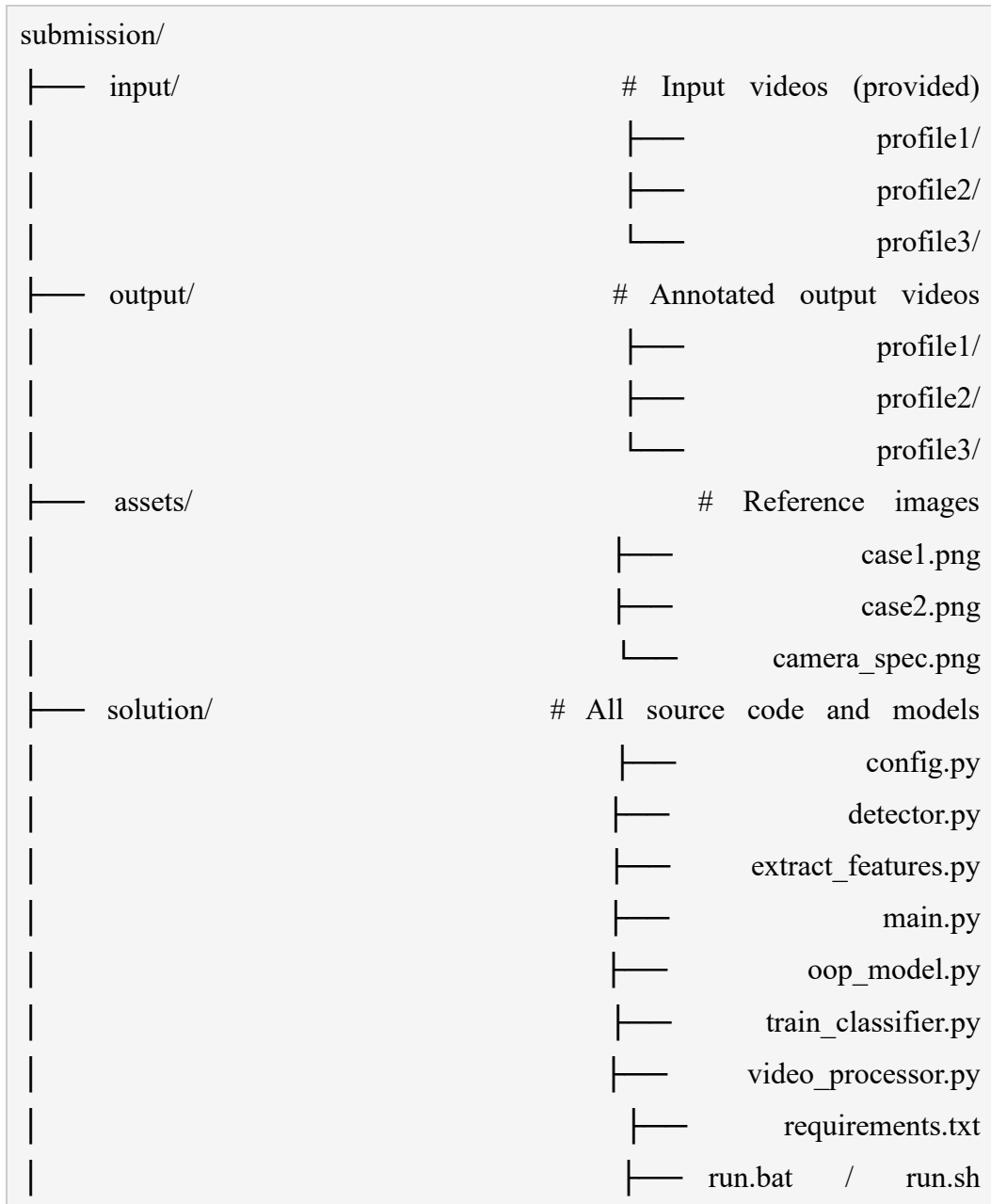
Table 4 Rule-based heuristic thresholds for the fallback detector.

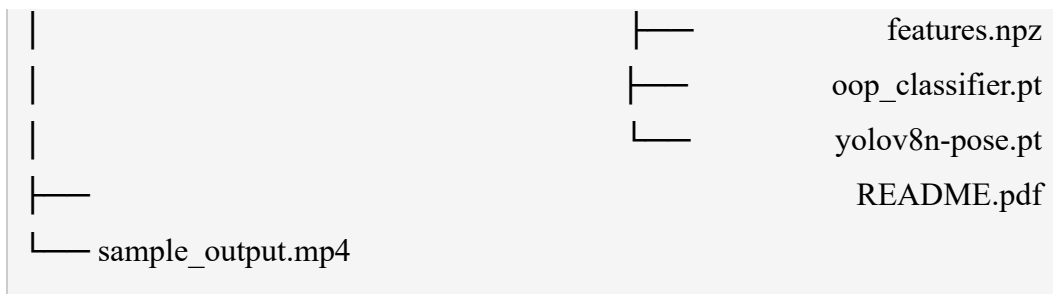
4.7 Temporal Smoothing - Hysteresis State Machine

Raw per-frame predictions can flicker the label may switch between POSITIVE and NEGATIVE across consecutive frames even when the passenger's posture has not changed, caused by brief drops in YOLO keypoint confidence on static scenes. A Hysteresis State Machine is applied after every frame prediction. Entering the POSITIVE state requires at least one confirmed positive frame within the last five frames, with the state switching immediately on the first confirmed detection. Exiting the POSITIVE state requires 20 consecutive negative frames (one full second at 20 fps) before switching back. The design is fast to detect OOP (reacting within one frame) and slow to release (requiring sustained evidence of normal posture before switching back)

5. PROJECT STRUCTURE

5.1 Folder Structure





5.2 File Descriptions

File	Purpose
config.py	Central settings: camera parameters, thresholds, model paths, display options.
detector.py	Core engine: preprocessing, YOLOv8 inference, passenger selection, OOP decision, frame rendering.
oop_model.py	LSTM model definition and feature extraction shared by training and inference.
extract_features.py	Offline script runs YOLOv8 on all input videos and saves keypoints to features.npz.
train_classifier.py	Offline script trains the Bidirectional LSTM on features.npz and saves oop_classifier.pt.
video_processor.py	Video I/O manager reads input videos, calls detector frame by frame, writes output.

main.py	Entry point reads command-line arguments and launches the full detection pipeline.
requirements.txt	Python dependency list with minimum version numbers.
run.bat / run.sh	One-click runners for Windows and Linux/macOS automate all four steps end to end.
features.npz	Pre-extracted keypoint feature dataset skips re-extraction on first run.
oop_classifier.pt	Pre-trained LSTM model weights skips retraining on first run.
yolov8n-pose.pt	YOLOv8-Pose model weights auto-downloaded on first run if not present.

6. HOW TO RUN - REPRODUCTION GUIDE

6.1 Requirements

Requirement	Detail
Operating System	Windows 10/11, Ubuntu 20.04+, macOS 12+
Python Version	3.10 or later
Internet Connection	Required on first run only (to auto-download yolov8n-pose.pt ~6 MB)
GPU	Not required: CUDA is used automatically if available, otherwise CPU is used
RAM	Minimum 4 GB recommended
Disk Space	Minimum 2 GB free (for virtual environment and model files)

6.2 Quick Start - One Click (Recommended)

Place the input videos in the input/ folder following the structure shown in Chapter 5, then run the appropriate script from inside the solution/ folder:

```
#Windows
run.bat

#Linux/macOS
chmod +x run.sh (only needed once)
# ./run.sh
```

The script will automatically create a Python virtual environment, install all dependencies, extract keypoint features, train the LSTM classifier, and write annotated output videos to the output/ folder.

6.3 Manual Step-by-Step (Alternative)

```
# Step 1: Install dependencies
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt

# Step 2: Extract features
.venv/bin/python extract_features.py

# Step 3: Train the classifier
.venv/bin/python train_classifier.py --epochs 80 --val-ratio 0

# Step 4: Run the detector
.venv/bin/python main.py
```

6.4 Custom Paths (Optional)

All scripts support custom input and output paths via command-line arguments:

```
python main.py --input path/to/input --output path/to/output
python extract_features.py --input path/to/input --output path/to/features.npz
python train_classifier.py --features path/to/features.npz --out path/to/model.pt
--epochs 80
```

6.5 Using a Larger Model (Optional)

By default the system uses yolov8n-pose.pt (nano: fastest). For better accuracy at the cost of speed, a larger model can be specified:

```
python main.py --model yolov8s-pose.pt
python main.py --model yolov8m-pose.pt
python main.py --model yolov8l-pose.pt
```

6.6 Expected Output

After successful execution, annotated .mp4 files are written to the output/ folder, mirroring the input folder structure. Each output video carries a colour-coded banner on every frame: a green banner indicates normal posture (NEGATIVE), a red banner indicates foot on dashboard (POSITIVE), and a grey banner indicates no detection (UNKNOWN). A scrolling timeline graph at the bottom of each frame shows the detection history across the last 300 frames.

7. RESULTS AND KEY FINDINGS

7.1 Output Video Summary

The system successfully processes all input videos across all three profiles and produces a fully annotated output video for each one. Every frame carries one of three labels {POSITIVE, NEGATIVE, or UNKNOWN} displayed as a colour-coded banner at the top of the frame with a scrolling timeline graph at the bottom.

7.2 Detection Behaviour Observed

On negative videos, the system correctly maintains the NEGATIVE label throughout the entire video. The temporal state machine holds the state steadily with no false flickers. UNKNOWN frames are rare and occur only when YOLO momentarily loses the passenger due to motion blur or extreme body angles. On positive videos, the system correctly detects the transition from NEGATIVE to POSITIVE when the foot is raised onto the dashboard, holds the POSITIVE state stably even when YOLO briefly loses confidence on the ankle keypoint during a static OOP posture, and correctly returns to NEGATIVE once the foot is lowered and 20 consecutive negative frames are confirmed.

7.3 Key Findings

Finding 1: Ankle elevation alone is insufficient. A simple threshold on ankle height produces too many false positives. Passengers shifting in their seat, crossing their legs, or leaning forward can temporarily raise the ankle to a height similar to the OOP posture. The LSTM learns to distinguish these cases by examining the full body posture pattern over 16 frames.

Finding 2: The fisheye lens does not require correction for the task. After testing with and without lens distortion correction, the body keypoint detection

quality was not meaningfully improved by undistortion. The fisheye distortion affects the edges of the frame more than the central cabin area where the passenger sits, and applying correction introduces black borders that reduce the usable frame area.

Finding 3: The state machine is essential for stable output. Without the hysteresis state machine, the raw per-frame LSTM output flickers between POSITIVE and NEGATIVE on static scenes where the ankle is on the dashboard but keypoint confidence drops slightly. The state machine eliminates completely.

Finding 4: Passenger identification is reliable. The rightmost hip-centre selection correctly identifies the front passenger in all tested videos. The driver consistently appears on the left side of the frame and the passenger on the right.

Finding 5: The heuristic fallback is a useful safety net. When tested without the trained LSTM model, the rule-based heuristic correctly identifies clear OOP postures where the ankle is well above the dashboard line. It struggles with borderline cases, which is expected, as those are precisely what the LSTM was trained to handle.

7.4 Where the System Works Well

- Clear, unambiguous OOP postures where both ankles are visibly raised and resting on the dashboard.
- Videos where the passenger is well lit by the IR LED and keypoints are detected with high confidence.
- Transitions between normal and OOP posture, the system catches the moment of change quickly.

- Static OOP postures held for several seconds the state machine maintains the POSITIVE label reliably.

7.5 Where the System Struggles

- Partial OOP postures only one foot raised or foot resting at the edge of the dashboard zone.
- Heavily occluded ankles when the passenger's clothing or seat position hides the ankle from the camera.
- Very brief OOP postures a foot placed on the dashboard for fewer than five frames may not trigger the state machine.
- Unusual body angles passengers leaning sideways or turned away from the camera reduce keypoint detection quality.

8. ASSUMPTIONS MADE

The following assumptions were made during the design and development of the solution, based on the assignment description, the camera specifications, and observations from the input videos.

Assumption 1 The front passenger always sits on the right side of the frame.

The camera is mounted at the rear-view mirror position in a left-hand drive (LHD) vehicle. In the configuration, the driver appears on the left side of the frame and the front passenger appears on the right. The passenger identification strategy is built on the assumption and would not work correctly for a right-hand drive vehicle without changing the selection logic.

Assumption 2 All videos are from the same camera and vehicle. The camera intrinsic parameters and dashboard threshold values are calibrated for the specific Intensirics IR camera and Hyundai Santa Fe used in the work. The system is not guaranteed to function correctly with a different camera or vehicle cabin geometry without recalibration.

Assumption 3 Normal posture means both feet are at floor level. All negative videos are treated as ground truth for normal posture, meaning every frame in a negative video is labelled NEGATIVE during training. The assumes that passengers in negative videos always have both feet on the floor.

Assumption 4 Positive videos contain OOP posture in at least some frames. The system uses a per-frame pseudo-label during training based on ankle position rather than labelling every frame in a positive video as POSITIVE. This prevents the LSTM from learning to identify which video it is watching rather than what OOP posture looks like.

Assumption 5 The IR image does not require colour information. The camera outputs a single-channel infrared grayscale image. Replicating the channel across all three BGR channels is sufficient for YOLOv8-Pose to detect body keypoints accurately.

Assumption 6 20 fps is sufficient temporal resolution. The state machine and sliding window parameters are calibrated for 20 fps video. If the video frame rate were significantly different, these parameters would need to be adjusted.

Assumption 7 The pre-trained model generalises across profiles. The LSTM classifier is trained on features extracted from all available input videos across all profiles. Body-relative normalisation makes features independent of body size and camera distance, supporting generalisation.

9. LIMITATIONS

Limitation 1 OOP Case 1 Only. The solution addresses only OOP Case 1 Feet on Dashboard. Extending the system to cover Case 2 (leaning forward) would require additional training data and a different set of body keypoint features focused on upper body and head position.

Limitation 2 Limited Training Data. The LSTM classifier is trained on a small dataset of 3 profiles and 11 videos. While body-relative normalisation and the HAR approach help the model generalise, a larger and more diverse dataset would significantly improve accuracy and robustness.

Limitation 3 Single Camera Setup. The system is calibrated and tested for one specific camera mounted in one specific vehicle. Using a different camera, mounting position, or vehicle would require recalibration of the threshold values in config.py.

Limitation 4 Dependence on YOLOv8 Keypoint Quality. The entire pipeline depends on YOLOv8-Pose producing reliable keypoint detections. When the passenger is in an unusual body position, partially occluded, or at the edge of the frame, low-confidence or missing keypoints may cause detection gaps.

Limitation 5 No Real-Time Optimisation. The system processes videos offline and has not been optimised for real-time inference. On a CPU, processing speed may fall below 20 fps for larger YOLOv8 models.

Limitation 6 Fixed Temporal Parameters. The sliding window size (16 frames) and the release threshold (20 frames) are calibrated for 20 fps video. A video at a different frame rate would require manual adjustment of these parameters.

Limitation 7 No Confidence Score in Output. The system outputs a binary label per frame without a probability score. A per-frame confidence value would provide downstream systems with more granular information for warning thresholds.

Limitation 8 One-Click Runner Portability. The `run.sh` script for Linux and macOS has not been tested across all distributions and versions. Minor adjustments may be needed depending on the system's Python installation.

10. FUTURE IMPROVEMENTS

10.1 Extend to OOP Case 2 - Leaning Forward

The most immediate next step is to extend the system to detect OOP Case 2, where the passenger leans forward and is in close proximity to the dashboard. It would require collecting and labelling training data for Case 2 postures, adding upper body and head keypoint features to the feature vector, and training a separate classifier or extending the existing LSTM to handle both cases simultaneously.

10.2 Larger YOLOv8 Model for Better Keypoint Accuracy

The current solution uses yolov8n-pose.pt (nano) the smallest and fastest model. Upgrading to yolov8s-pose.pt or yolov8m-pose.pt would produce more accurate keypoint detections, particularly for difficult frames where the passenger is partially occluded or at an unusual angle. YOLO is a one-line change in config.py and requires no code modification.

10.3 Real-Time Inference Support

Converting the system to a real-time pipeline would make it suitable for deployment in an actual vehicle. It involves reading frames directly from a live camera feed, optimising LSTM inference using TorchScript or ONNX export, and running YOLOv8 with TensorRT on an embedded GPU to achieve sustained throughput of 20 fps.

10.4 Larger and More Diverse Training Dataset

Training on only 11 videos limits the model's ability to generalise. Additional data covering passengers with different body types and heights, varied seating

positions, more vehicle models, and different IR lighting conditions would significantly reduce false positives on edge cases.

10.5 Confidence Score Output

Adding a per-frame probability score (0.0 to 1.0) alongside the binary label would make the system more useful in production, allowing downstream systems to apply different response thresholds for soft and hard warnings.

10.6 Right-Hand Drive Vehicle Support

A configurable `VEHICLE_SIDE` flag in `config.py` that switches the passenger selection logic from rightmost to leftmost hip centre would make the system compatible with right-hand drive markets (United Kingdom, Japan, India, Australia) without any code changes.

10.7 Automatic Threshold Calibration

The dashboard threshold values are currently set manually based on camera geometry. An automatic calibration step where the passenger sits normally and places their foot on the dashboard once would derive these values from a short calibration video, making the system portable across different camera positions and vehicles without manual tuning.

10.8 Multi-Passenger Support

The current system tracks only the front passenger. Extending it to simultaneously monitor both the driver and the front passenger each with their own independent OOP state machine would significantly increase the safety coverage of the system.

REMARKS

As the ReadME document mentioned, the threshold and the values were not adjusted to achieve results for specific videos and cases, it has all been set to global and based on the approach the project was done. Adjusting the values for specific cases might help globally because they are the real cases and data that moulds the output together to achieve a A'star application/model. The skeleton, joints are only processed in the process and does not display it in the output videos, to provide a much cleaner output with a line graph below and right corner the frames running can be observed.

REFERENCES

- [1] Euro NCAP (2024). Euro NCAP Assessment Protocol — Occupant Status Monitoring. European New Car Assessment Programme. <https://www.euroncap.com/en/for-engineers/protocols/safety-assist/>
- [2] Jocher, G., et al. (2023). Ultralytics YOLOv8 — Pose Estimation. Ultralytics Inc. <https://docs.ultralytics.com/tasks/pose/>
- [3] Lin, T. Y., et al. (2014). Microsoft COCO: Common Objects in Context. European Conference on Computer Vision. <https://cocodataset.org/#keypoints-2020>
- [4] PyTorch Foundation (2024). torch.nn.LSTM. PyTorch Documentation. <https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html>
- [5] OpenCV Documentation (2024). Camera Calibration and 3D Reconstruction. OpenCV 4.x. https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html
- [6] Ordóñez, F. J. & Roggen, D. (2016). Deep Convolutional and LSTM Recurrent Neural Networks for Multimodal Wearable Activity Recognition. *Sensors*, 16(1), 115.
- [7] STMicroelectronics / GEO Semiconductor / OSRAM (2024). Intensirics IR Camera Hardware Specification. Provided as part of assignment supplemental material.
- [8] Brown, D. C. (1966). Decentering Distortion of Lenses. *Photometric Engineering*, 32(3), 444–4.